

Passing and returning COM smart pointers

A function taking a COM smart pointer in the form of a `winrt::com_ptr` should do so by constant reference, or by reference.

C++/WinRT

```
... GetDxgiFactory(winrt::com_ptr<ID3D11Device> const& device) ...  
  
... CreateDevice(..., winrt::com_ptr<ID3D11Device>& device) ...
```

A function that returns a `winrt::com_ptr` should do so by value.

C++/WinRT

```
winrt::com_ptr<ID2D1Factory1> CreateFactory() ...
```

Query a COM smart pointer for a different interface

You can use the `com_ptr::as` function to query a COM smart pointer for a different interface. The function throws an exception if the query doesn't succeed.

C++/WinRT

```
void ExampleFunction(winrt::com_ptr<ID3D11Device> const& device)  
{  
    ...  
    winrt::com_ptr<IDXGIDevice> const dxdevice{ device.as<IDXGIDevice>() };  
    ...  
}
```

Alternatively, use `com_ptr::try_as`, which returns a value that you can check against `nullptr` to see whether the query succeeded.

Full source code listing of a minimal Direct2D application

ⓘ Note

For info about setting up Visual Studio for C++/WinRT development—including installing and using the C++/WinRT Visual Studio Extension (VSIX) and the NuGet package (which together provide project template and build support)—see [Visual Studio support for C++/WinRT](#).

If you want to build and run this source code example then first install (or update to) the latest version of the C++/WinRT Visual Studio Extension (VSIX); see the note above. Then, in Visual Studio, create a new **Core App (C++/WinRT)**. `Direct2D` is a reasonable name for the project, but you can name it anything you like. Target the latest generally-available (that is, not preview) version of the Windows SDK.

Step 1. Edit `pch.h`

Open `pch.h`, and add `#include <unkwn.h>` immediately after including `windows.h`. This is because we're using `winrt::get_unknown`. It's a good idea to `#include <unkwn.h>` explicitly whenever you use `winrt::get_unknown`, even if that header has been included by another header.

ⓘ Note

If you omit this step then you'll see the build error `'get_unknown': identifier not found`.

Step 2. Edit `App.cpp`

Open `App.cpp`, delete its entire contents, and paste in the listing below.

The code below uses the `winrt::com_ptr::capture function` where possible. `WINRT_ASSERT` is a macro definition, and it expands to `_ASSERTE`.

C++/WinRT

```
#include "pch.h"
#include <d2d1_1.h>
#include <d3d11.h>
#include <dxgi1_2.h>
#include <winrt/Windows.Graphics.Display.h>

using namespace winrt;

using namespace Windows;
using namespace Windows::ApplicationModel::Core;
using namespace Windows::UI;
```

```

using namespace Windows::UI::Core;
using namespace Windows::Graphics::Display;

namespace
{
    winrt::com_ptr<ID2D1Factory1> CreateFactory()
    {
        D2D1_FACTORY_OPTIONS options{};

#ifdef _DEBUG
        options.debugLevel = D2D1_DEBUG_LEVEL_INFORMATION;
#endif

        winrt::com_ptr<ID2D1Factory1> factory;

        winrt::check_hresult(D2D1CreateFactory(
            D2D1_FACTORY_TYPE_SINGLE_THREADED,
            options,
            factory.put()));

        return factory;
    }

    HRESULT CreateDevice(D3D_DRIVER_TYPE const type,
        winrt::com_ptr<ID3D11Device>& device)
    {
        WINRT_ASSERT(!device);

        return D3D11CreateDevice(
            nullptr,
            type,
            nullptr,
            D3D11_CREATE_DEVICE_BGRA_SUPPORT,
            nullptr, 0,
            D3D11_SDK_VERSION,
            device.put(),
            nullptr,
            nullptr);
    }

    winrt::com_ptr<ID3D11Device> CreateDevice()
    {
        winrt::com_ptr<ID3D11Device> device;
        HRESULT hr{ CreateDevice(D3D_DRIVER_TYPE_HARDWARE, device) };

        if (DXGI_ERROR_UNSUPPORTED == hr)
        {
            hr = CreateDevice(D3D_DRIVER_TYPE_WARP, device);
        }

        winrt::check_hresult(hr);
        return device;
    }

    winrt::com_ptr<ID2D1DeviceContext> CreateRenderTarget(

```

```

        winrt::com_ptr<ID2D1Factory1> const& factory,
        winrt::com_ptr<ID3D11Device> const& device)
    {
        WINRT_ASSERT(factory);
        WINRT_ASSERT(device);

        winrt::com_ptr<IDXGIDevice> const dxdevice{ device.as<IDXGIDevice>()
};

        winrt::com_ptr<ID2D1Device> d2device;
        winrt::check_hresult(factory->CreateDevice(dxdevice.get(),
d2device.put()));

        winrt::com_ptr<ID2D1DeviceContext> target;
        winrt::check_hresult(d2device-
>CreateDeviceContext(D2D1_DEVICE_CONTEXT_OPTIONS_NONE, target.put()));
        return target;
    }

    winrt::com_ptr<IDXGIFactory2>
    GetDxgiFactory(winrt::com_ptr<ID3D11Device> const& device)
    {
        WINRT_ASSERT(device);

        winrt::com_ptr<IDXGIDevice> const dxdevice{ device.as<IDXGIDevice>()
};

        winrt::com_ptr<IDXGIAdapter> adapter;
        winrt::check_hresult(dxdevice->GetAdapter(adapter.put()));

        winrt::com_ptr<IDXGIFactory2> factory;
        factory.capture(adapter, &IDXGIAdapter::GetParent);
        return factory;
    }

    void CreateDeviceSwapChainBitmap(
        winrt::com_ptr<IDXGISwapChain1> const& swapchain,
        winrt::com_ptr<ID2D1DeviceContext> const& target)
    {
        WINRT_ASSERT(swapchain);
        WINRT_ASSERT(target);

        winrt::com_ptr<IDXGISurface> surface;
        surface.capture(swapchain, &IDXGISwapChain1::GetBuffer, 0);

        D2D1_BITMAP_PROPERTIES1 const props{ D2D1::BitmapProperties1(
            D2D1_BITMAP_OPTIONS_TARGET | D2D1_BITMAP_OPTIONS_CANNOT_DRAW,
            D2D1::PixelFormat(DXGI_FORMAT_B8G8R8A8_UNORM,
D2D1_ALPHA_MODE_IGNORE)) };

        winrt::com_ptr<ID2D1Bitmap1> bitmap;

        winrt::check_hresult(target-
>CreateBitmapFromDxgiSurface(surface.get(),
            props,

```

```

        bitmap.put()));

    target->SetTarget(bitmap.get());
}

winrt::com_ptr<IDXGISwapChain1>
CreateSwapChainForCoreWindow(winrt::com_ptr<ID3D11Device> const& device)
{
    WINRT_ASSERT(device);

    winrt::com_ptr<IDXGIFactory2> const factory{ GetDxgiFactory(device)
};

    DXGI_SWAP_CHAIN_DESC1 props{};
    props.Format = DXGI_FORMAT_B8G8R8A8_UNORM;
    props.SampleDesc.Count = 1;
    props.BufferUsage = DXGI_USAGE_RENDER_TARGET_OUTPUT;
    props.BufferCount = 2;
    props.SwapEffect = DXGI_SWAP_EFFECT_FLIP_SEQUENTIAL;

    winrt::com_ptr<IDXGISwapChain1> swapChain;

    winrt::check_hresult(factory->CreateSwapChainForCoreWindow(
        device.get(),
        winrt::get_unknown(CoreWindow::GetForCurrentThread()),
        &props,
        nullptr, // all or nothing
        swapChain.put()));

    return swapChain;
}

constexpr D2D1_COLOR_F color_white{ 1.0f, 1.0f, 1.0f, 1.0f };
constexpr D2D1_COLOR_F color_orange{ 0.92f, 0.38f, 0.208f, 1.0f };
}

struct App : implements<App, IFrameworkViewSource, IFrameworkView>
{
    winrt::com_ptr<ID2D1Factory1> m_factory;
    winrt::com_ptr<ID2D1DeviceContext> m_target;
    winrt::com_ptr<IDXGISwapChain1> m_swapChain;
    winrt::com_ptr<ID2D1SolidColorBrush> m_brush;
    float m_dpi{};

    IFrameworkView CreateView()
    {
        return *this;
    }

    void Initialize(CoreApplicationView const&)
    {
    }

    void Load(hstring const&)
    {
    }

```

```

CoreWindow const window{ CoreWindow::GetForCurrentThread() };

window.SizeChanged([&](auto&&...)
{
    if (m_target)
    {
        ResizeSwapChainBitmap();
        Render();
    }
});

DisplayInformation const display{
DisplayInformation::GetForCurrentView() };
m_dpi = display.LogicalDpi();

display.DpiChanged([&](DisplayInformation const& display,
IInspectable const&)
{
    if (m_target)
    {
        m_dpi = display.LogicalDpi();
        m_target->SetDpi(m_dpi, m_dpi);
        CreateDeviceSizeResources();
        Render();
    }
});

m_factory = CreateFactory();
CreateDeviceIndependentResources();
}

void Uninitialize()
{
}

void Run()
{
    CoreWindow const window{ CoreWindow::GetForCurrentThread() };
    window.Activate();

    Render();
    CoreDispatcher const dispatcher{ window.Dispatcher() };
    dispatcher.ProcessEvents(CoreProcessEventsOption::ProcessUntilQuit);
}

void SetWindow(CoreWindow const&) {}

void Draw()
{
    m_target->Clear(color_white);

    D2D1_SIZE_F const size{ m_target->GetSize() };
    D2D1_RECT_F const rect{ 100.0f, 100.0f, size.width - 100.0f,
size.height - 100.0f };
    m_target->DrawRectangle(rect, m_brush.get(), 100.0f);
}

```

```

        char buffer[1024];
        (void)snprintf(buffer, sizeof(buffer), "Draw %.2f x %.2f @ %.2f\n",
size.width, size.height, m_dpi);
        ::OutputDebugStringA(buffer);
    }

void Render()
{
    if (!m_target)
    {
        winrt::com_ptr<ID3D11Device> const device{ CreateDevice() };
        m_target = CreateRenderTarget(m_factory, device);
        m_swapChain = CreateSwapChainForCoreWindow(device);

        CreateDeviceSwapChainBitmap(m_swapChain, m_target);

        m_target->SetDpi(m_dpi, m_dpi);

        CreateDeviceResources();
        CreateDeviceSizeResources();
    }

    m_target->BeginDraw();
    Draw();
    m_target->EndDraw();

    HRESULT const hr{ m_swapChain->Present(1, 0) };

    if (S_OK != hr && DXGI_STATUS_OCCLUDED != hr)
    {
        ReleaseDevice();
    }
}

void ReleaseDevice()
{
    m_target = nullptr;
    m_swapChain = nullptr;

    ReleaseDeviceResources();
}

void ResizeSwapChainBitmap()
{
    WINRT_ASSERT(m_target);
    WINRT_ASSERT(m_swapChain);

    m_target->SetTarget(nullptr);

    if (S_OK == m_swapChain->ResizeBuffers(0, // all buffers
        0, 0, // client area
        DXGI_FORMAT_UNKNOWN, // preserve format
        0)) // flags
    {

```

```

        CreateDeviceSwapChainBitmap(m_swapChain, m_target);
        CreateDeviceSizeResources();
    }
    else
    {
        ReleaseDevice();
    }
}

void CreateDeviceIndependentResources()
{
}

void CreateDeviceResources()
{
    winrt::check_hresult(m_target->CreateSolidColorBrush(
        color_orange,
        D2D1::BrushProperties(0.8f),
        m_brush.put()));
}

void CreateDeviceSizeResources()
{
}

void ReleaseDeviceResources()
{
    m_brush = nullptr;
}
};

int __stdcall wWinMain(HINSTANCE, HINSTANCE, PWSTR, int)
{
    CoreApplication::Run(winrt::make<App>());
}

```

Working with COM types, such as BSTR and VARIANT

As you can see, C++/WinRT provides support for both implementing and calling COM interfaces. For using COM types, such as BSTR and VARIANT, we recommend that you use wrappers provided by the [Windows Implementation Libraries \(WIL\)](#), such as `wil::unique_bstr` and `wil::unique_variant` (which manage resources lifetimes).

[WIL](#) supersedes frameworks such as the Active Template Library (ATL), and the Visual C++ compiler's COM Support. And we recommend it over writing your own wrappers, or using COM types such as BSTR and VARIANT in their raw form (together with the appropriate APIs).

Avoiding namespace collisions

It's common practice in C++/WinRT—as the code listing in this topic demonstrates—to use using-directives liberally. In some cases, though, that can lead to the problem of importing colliding names into the global namespace. Here's an example.

C++/WinRT contains a type named `winrt::Windows::Foundation::IUnknown`; while COM defines a type named `::IUnknown`. So consider the following code, in a C++/WinRT project that consumes COM headers.

C++/WinRT

```
using namespace winrt::Windows::Foundation;
...
void MyFunction(IUnknown*); // error C2872: 'IUnknown': ambiguous symbol
```

The unqualified name *IUnknown* collides in the global namespace, hence the *ambiguous symbol* compiler error. Instead, you can isolate the C++/WinRT version of the name into the `winrt` namespace, like this.

C++/WinRT

```
namespace winrt
{
    using namespace Windows::Foundation;
}
...
void MyFunctionA(IUnknown*); // Ok.
void MyFunctionB(winrt::IUnknown const&); // Ok.
```

Or, if you want the convenience of `using namespace winrt`, then you can. You just need to qualify the global version of *IUnknown*, like this.

C++/WinRT

```
using namespace winrt;
namespace winrt
{
    using namespace Windows::Foundation;
}
...
void MyFunctionA(::IUnknown*); // Ok.
void MyFunctionB(winrt::IUnknown const&); // Ok.
```

Naturally, this works with any C++/WinRT namespace.

```
namespace winrt
{
    using namespace Windows::Storage;
    using namespace Windows::System;
}
```

You can then refer to `winrt::Windows::Storage::StorageFile`, for example, as just `winrt::StorageFile`.

Important APIs

- [winrt::check_hresult](#) function
- [winrt::com_ptr](#) struct template
- [winrt::Windows::Foundation::IUnknown](#) struct

Feedback

Was this page helpful?



[Provide product feedback](#)  | [Get help at Microsoft Q&A](#)

Author COM components with C++/WinRT

Article • 10/20/2022

C++/WinRT can help you to author classic Component Object Model (COM) components (or coclasses), just as it helps you to author Windows Runtime classes. This topic shows you how.

How C++/WinRT behaves, by default, with respect to COM interfaces

C++/WinRT's `winrt::implements` template is the base from which your runtime classes and activation factories directly or indirectly derive.

By default, `winrt::implements` silently ignores classic COM interfaces. Any `QueryInterface` (QI) calls for classic COM interfaces will consequently fail with `E_NOINTERFACE`. By default, `winrt::implements` supports only C++/WinRT interfaces.

- `winrt::IUnknown` is a C++/WinRT interface, so `winrt::implements` supports `winrt::IUnknown`-based interfaces.
- `winrt::implements` doesn't support `::IUnknown` itself, by default.

In a moment you'll see how to overcome the cases that aren't supported by default. But first here's a code example to illustrate what happens by default.

idl

```
// Sample.idl
namespace MyProject
{
    runtimeclass Sample
    {
        Sample();
        void DoWork();
    }
}

// Sample.h
#include "pch.h"
#include <shobjidl.h> // Needed only for this file.

namespace winrt::MyProject::implementation
{
    struct Sample : implements<Sample, IInitializeWithWindow>
```

```

{
    IFACEMETHOD(Initialize)(HWND hwnd);
    void DoWork();
}
}

```

And here's client code to consume the **Sample** class.

C++/WinRT

```

// Client.cpp
Sample sample; // Construct a Sample object via its projection.

// This next line doesn't compile yet.
sample.as<IInitializeWithWindow>()->Initialize(hwnd);

```

Enabling classic COM support

The good news is that all it takes to cause **winrt::implements** to support classic COM interfaces is to include the `unknwn.h` header file before you include any C++/WinRT headers.

You could do that explicitly, or indirectly by including some other header file such as `ole2.h`. One recommended method is to include the `wil\cppwinrt.h` header file, which is part of the [Windows Implementation Libraries \(WIL\)](#) [↗](#). The `wil\cppwinrt.h` header file not only makes sure that `unknwn.h` is included before `winrt/base.h`, it also sets things up so that C++/WinRT and WIL understand each other's exceptions and error codes.

You can then `as<>` for classic COM interfaces, and the code in the example above will compile.

ⓘ Note

In the example above, even after enabling classic COM support in the client (the code consuming the class), if you haven't also enabled classic COM support in the server (the code implementing the class), then the call to `as<>` in the client will crash because the QI for `IInitializeWithWindow` will fail.

A local (non-projected) class

A local class is one that is both implemented and consumed in the same compilation unit (app, or other binary); and so there is no projection for it.

Here's an example of a local class that implements *only classic COM interfaces*.

C++/WinRT

```
struct LocalObject :  
    winrt::implements<LocalObject, IInitializeWithWindow>  
{  
    ...  
};
```

If you implement that example, but you don't enable classic COM support, then the following code fails.

C++/WinRT

```
winrt::make<LocalObject>(); // error: 'first_interface': is not a member of  
    'winrt::impl::interface_list<>'
```

Again, `IInitializeWithWindow` isn't recognized as a COM interface, so C++/WinRT ignores it. In the case of the `LocalObject` example, the result of ignoring COM interfaces means that `LocalObject` has no interfaces at all. But every COM class must implement at least one interface.

A simple example of a COM component

Here's a simple example of a COM component written using C++/WinRT. This is a full listing of a mini-application, so you can try the code out if you paste it into the `pch.h` and `main.cpp` of a new **Windows Console Application (C++/WinRT)** project.

C++/WinRT

```
// pch.h  
#pragma once  
#include <unkwn.h>  
#include <winrt/Windows.Foundation.h>  
  
// main.cpp : Defines the entry point for the console application.  
#include "pch.h"  
  
struct __declspec(uuid("ddc36e02-18ac-47c4-ae17-d420eece2281"))  
    IMyComInterface : ::IUnknown  
{  
    virtual HRESULT __stdcall Call() = 0;  
};  
  
using namespace winrt;  
using namespace Windows::Foundation;
```

```

int main()
{
    winrt::init_apartment();

    struct MyCoclass : winrt::implements<MyCoclass, IPersist, IStringable,
IMyComInterface>
    {
        HRESULT __stdcall Call() noexcept override
        {
            return S_OK;
        }

        HRESULT __stdcall GetClassID(CLSID* id) noexcept override
        {
            *id = IID_IPersist; // Doesn't matter what we return, for this
example.
            return S_OK;
        }

        winrt::hstring ToString()
        {
            return L"MyCoclass as a string";
        }
    };

    auto mycoclass_instance{ winrt::make<MyCoclass>() };
    CLSID id{};
    winrt::check_hresult(mycoclass_instance->GetClassID(&id));
    winrt::check_hresult(mycoclass_instance.as<IMyComInterface>()->Call());
}

```

Also see [Consume COM components with C++/WinRT](#).

A more realistic and interesting example

The remainder of this topic walks through creating a minimal console application project that uses C++/WinRT to implement a basic coclass (COM component, or COM class) and class factory. The example application shows how to deliver a toast notification with a callback button on it, and the coclass (which implements the **INotificationActivationCallback** COM interface) allows the application to be launched and called back when the user clicks that button on the toast.

More background about the toast notification feature area can be found at [Send a local toast notification](#). None of the code examples in that section of the documentation use C++/WinRT, though, so we recommend that you prefer the code shown in this topic.

Create a Windows Console Application project (ToastAndCallback)

Begin by creating a new project in Microsoft Visual Studio. Create a **Windows Console Application (C++/WinRT)** project, and name it *ToastAndCallback*.

Open `pch.h`, and add `#include <unkwn.h>` before the includes for any C++/WinRT headers. Here's the result; you can replace the contents of your `pch.h` with this listing.

C++/WinRT

```
// pch.h
#pragma once
#include <unkwn.h>
#include <winrt/Windows.Foundation.h>
```

Open `main.cpp`, and remove the using-directives that the project template generates. In their place, insert the following code (which gives us the libs, headers, and type names that we need). Here's the result; you can replace the contents of your `main.cpp` with this listing (we've also removed the code from `main` in the listing below, because we'll be replacing that function later).

C++/WinRT

```
// main.cpp : Defines the entry point for the console application.

#include "pch.h"

#pragma comment(lib, "advapi32")
#pragma comment(lib, "ole32")
#pragma comment(lib, "shell32")

#include <iomanip>
#include <iostream>
#include <notificationactivationcallback.h>
#include <propkey.h>
#include <propvarutil.h>
#include <shlobj.h>
#include <winrt/Windows.UI.Notifications.h>
#include <winrt/Windows.Data.Xml.Dom.h>

using namespace winrt;
using namespace Windows::Data::Xml::Dom;
using namespace Windows::UI::Notifications;

int main() { }
```

The project won't build yet; after we've finished adding code, you'll be prompted to build and run.

Implement the coclass and class factory

In C++/WinRT, you implement coclasses, and class factories, by deriving from the `winrt::implements` base struct. Immediately after the three using-directives shown above (and before `main`), paste this code to implement your toast notification COM activator component.

C++/WinRT

```
static constexpr GUID callback_guid // BAF2FA85-E121-4CC9-A942-CE335B6F917F
{
    0xBAF2FA85, 0xE121, 0x4CC9, {0xA9, 0x42, 0xCE, 0x33, 0x5B, 0x6F, 0x91,
    0x7F}
};

std::wstring const this_app_name{ L"ToastAndCallback" };

struct callback : winrt::implements<callback,
INotificationActivationCallback>
{
    HRESULT __stdcall Activate(
        LPCWSTR app,
        LPCWSTR args,
        [[maybe_unused]] NOTIFICATION_USER_INPUT_DATA const* data,
        [[maybe_unused]] ULONG count) noexcept final
    {
        try
        {
            std::wcout << this_app_name << L" has been called back from a
notification." << std::endl;
            std::wcout << L"Value of the 'app' parameter is '" << app <<
L"'." << std::endl;
            std::wcout << L"Value of the 'args' parameter is '" << args <<
L"'." << std::endl;
            return S_OK;
        }
        catch (...)
        {
            return winrt::to_hresult();
        }
    }
};

struct callback_factory : implements<callback_factory, IClassFactory>
{
    HRESULT __stdcall CreateInstance(
        IUnknown* outer,
        GUID const& iid,
```



```

        void** result) noexcept final
    {
        *result = nullptr;

        if (outer)
        {
            return CLASS_E_NOAGGREGATION;
        }

        return make<callback>()->QueryInterface(iid, result);
    }

    HRESULT __stdcall LockServer(BOOL) noexcept final
    {
        return S_OK;
    }
};

```

The implementation of the coclass above follows the same pattern that's demonstrated in [Author APIs with C++/WinRT](#). So, you can use the same technique to implement COM interfaces as well as Windows Runtime interfaces. COM components and Windows Runtime classes expose their features via interfaces. Every COM interface ultimately derives from the [IUnknown interface](#). The Windows Runtime is based on COM—one distinction being that Windows Runtime interfaces ultimately derive from the [IInspectable interface](#) (and `IInspectable` derives from `IUnknown`).

In the coclass in the code above, we implement the `INotificationActivationCallback::Activate` method, which is the function that's called when the user clicks the callback button on a toast notification. But before that function can be called, an instance of the coclass needs to be created, and that's the job of the `IClassFactory::CreateInstance` function.

The coclass that we just implemented is known as the *COM activator* for notifications, and it has its class id (CLSID) in the form of the `callback_guid` identifier (of type **GUID**) that you see above. We'll be using that identifier later, in the form of a Start menu shortcut and a Windows Registry entry. The COM activator CLSID, and the path to its associated COM server (which is the path to the executable that we're building here) is the mechanism by which a toast notification knows what class to create an instance of when its callback button is clicked (whether the notification is clicked in Action Center or not).

Best practices for implementing COM methods

Techniques for error handling and for resource management can go hand-in-hand. It's more convenient and practical to use exceptions than error codes. And if you employ

the resource-acquisition-is-initialization (RAII) idiom, then you can avoid explicitly checking for error codes and then explicitly releasing resources. Such explicit checks make your code more convoluted than necessary, and it gives bugs plenty of places to hide. Instead, use RAII, and throw/catch exceptions. That way, your resource allocations are exception-safe, and your code is simple.

However, you mustn't allow exceptions to escape your COM method implementations. You can ensure that by using the `noexcept` specifier on your COM methods. It's ok for exceptions to be thrown anywhere in the call graph of your method, as long as you handle them before your method exits. If you use `noexcept`, but you then allow an exception to escape your method, then your application will terminate.

Add helper types and functions

In this step, we'll add some helper types and functions that the rest of the code makes use of. So, immediately before `main`, add the following.

C++/WinRT

```
struct prop_variant : PROPVARIANT
{
    prop_variant() noexcept : PROPVARIANT{}
    {
    }

    ~prop_variant() noexcept
    {
        clear();
    }

    void clear() noexcept
    {
        WINRT_VERIFY_(S_OK, ::PropVariantClear(this));
    }
};

struct registry_traits
{
    using type = HKEY;

    static void close(type value) noexcept
    {
        WINRT_VERIFY_(ERROR_SUCCESS, ::RegCloseKey(value));
    }

    static constexpr type invalid() noexcept
    {
        return nullptr;
    }
}
```

```

};

using registry_key = winrt::handle_type<registry_traits>;

std::wstring get_module_path()
{
    std::wstring path(100, L'?');
    uint32_t path_size{};
    DWORD actual_size{};

    do
    {
        path_size = static_cast<uint32_t>(path.size());
        actual_size = ::GetModuleFileName(nullptr, path.data(), path_size);

        if (actual_size + 1 > path_size)
        {
            path.resize(path_size * 2, L'?');
        }
    } while (actual_size + 1 > path_size);

    path.resize(actual_size);
    return path;
}

std::wstring get_shortcut_path()
{
    std::wstring format{ LR"(%ProgramData%\Microsoft\Windows\Start
Menu\Programs\)" };
    format += (this_app_name + L".lnk");

    auto required{ ::ExpandEnvironmentStrings(format.c_str(), nullptr, 0) };
    std::wstring path(required - 1, L'?');
    ::ExpandEnvironmentStrings(format.c_str(), path.data(), required);
    return path;
}

```

Implement the remaining functions, and the wmain entry point function

Delete your `main` function, and in its place paste this code listing, which includes code to register your coclass, and then to deliver a toast capable of calling back your application.

C++/WinRT

```

void register_callback()
{
    DWORD registration{};

```

```

winrt::check_hresult(::CoRegisterClassObject(
    callback_guid,
    make<callback_factory>().get(),
    CLSCTX_LOCAL_SERVER,
    REGCLS_SINGLEUSE,
    &registration));
}

void create_shortcut()
{
    auto link{ winrt::create_instance<IShellLink>(CLSID_ShellLink) };
    std::wstring module_path{ get_module_path() };
    winrt::check_hresult(link->SetPath(module_path.c_str()));

    auto store = link.as<IPropertyStore>();
    prop_variant value;
    winrt::check_hresult(::InitPropVariantFromString(this_app_name.c_str(),
&value));
    winrt::check_hresult(store->SetValue(PKEY_AppUserModel_ID, value));
    value.clear();
    winrt::check_hresult(::InitPropVariantFromCLSID(callback_guid, &value));
    winrt::check_hresult(store-
>SetValue(PKEY_AppUserModel_ToastActivatorCLSID, value));

    auto file{ store.as<IPersistFile>() };
    std::wstring shortcut_path{ get_shortcut_path() };
    winrt::check_hresult(file->Save(shortcut_path.c_str(), TRUE));

    std::wcout << L"In " << shortcut_path << L", created a shortcut to " <<
module_path << std::endl;
}

void update_registry()
{
    std::wstring key_path{ LR"(SOFTWARE\Classes\CLSID\{????????-????-????
-????-????????????})" };
    ::StringFromGUID2(callback_guid, key_path.data() + 23, 39);
    key_path += LR"(\LocalServer32)";
    registry_key key;

    winrt::check_win32(::RegCreateKeyEx(
        HKEY_CURRENT_USER,
        key_path.c_str(),
        0,
        nullptr,
        0,
        KEY_WRITE,
        nullptr,
        key.put(),
        nullptr));
    ::RegDeleteValue(key.get(), nullptr);

    std::wstring path{ get_module_path() };

    winrt::check_win32(::RegSetValueEx(

```

```

        key.get(),
        nullptr,
        0,
        REG_SZ,
        reinterpret_cast<BYTE const*>(path.c_str()),
        static_cast<uint32_t>((path.size() + 1) * sizeof(wchar_t))));

    std::wcout << L"In " << key_path << L", registered local server at " <<
path << std::endl;
}

void create_toast()
{
    XmlDocument xml;

    std::wstring toastPayload
    {
        LR"(
<toast>
  <visual>
    <binding template='ToastGeneric'>
      <text>)"
    };
    toastPayload += this_app_name;
    toastPayload += LR"(
      </text>
    </binding>
  </visual>
  <actions>
    <action content='Call back )";
    toastPayload += this_app_name;
    toastPayload += LR"(
    ' arguments='the_args' activationKind='Foreground' />
  </actions>
</toast>)"
    );
    xml.LoadXml(toastPayload);

    ToastNotification toast{ xml };
    ToastNotifier notifier{
ToastNotificationManager::CreateToastNotifier(this_app_name) };
    notifier.Show(toast);
    ::Sleep(50); // Give the callback chance to display.
}

void LaunchedNormally(HANDLE, INPUT_RECORD &, DWORD &);
void LaunchedFromNotification(HANDLE, INPUT_RECORD &, DWORD &);

int wmain(int argc, wchar_t * argv[], wchar_t * /* envp */[])
{
    winrt::init_apartment();

    register_callback();

    HANDLE consoleHandle{ ::GetStdHandle(STD_INPUT_HANDLE) };
    INPUT_RECORD buffer{};

```

```

DWORD events{};
::FlushConsoleInputBuffer(consoleHandle);

if (argc == 1)
{
    LaunchedNormally(consoleHandle, buffer, events);
}
else if (argc == 2 && wcscmp(argv[1], L"-Embedding") == 0)
{
    LaunchedFromNotification(consoleHandle, buffer, events);
}
}

void LaunchedNormally(HANDLE consoleHandle, INPUT_RECORD & buffer, DWORD &
events)
{
    try
    {
        bool runningAsAdmin{ ::IsUserAnAdmin() == TRUE };
        std::wcout << this_app_name << L" is running" << (runningAsAdmin ?
L" (administrator)." : L" (NOT as administrator).") << std::endl;

        if (runningAsAdmin)
        {
            create_shortcut();
            update_registry();
        }

        std::wcout << std::endl << L"Press 'T' to display a toast
notification (press any other key to exit)." << std::endl;

        ::ReadConsoleInput(consoleHandle, &buffer, 1, &events);
        if (towupper(buffer.Event.KeyEvent.uChar.UnicodeChar) == L'T')
        {
            create_toast();
        }
    }
    catch (winrt::hresult_error const& e)
    {
        std::wcout << L"Error: " << e.message().c_str() << L" (" << std::hex
<< std::showbase << std::setw(8) << static_cast<uint32_t>(e.code()) << L")"
<< std::endl;
    }
}

void LaunchedFromNotification(HANDLE consoleHandle, INPUT_RECORD & buffer,
DWORD & events)
{
    ::Sleep(50); // Give the callback chance to display its message.
    std::wcout << std::endl << L"Press any key to exit." << std::endl;
    ::ReadConsoleInput(consoleHandle, &buffer, 1, &events);
}

```

How to test the example application

Build the application, and then run it at least once as an administrator to cause the registration, and other setup, code to run. One way to do that is to run Visual Studio as an administrator, and then run the app from Visual Studio. Right-click Visual Studio in the taskbar to display the jump list, right-click Visual Studio on the jump list, and then click **Run as administrator**. Agree to the prompt, and then open the project. When you run the application, a message is displayed indicating whether or not the application is running as an administrator. If it isn't, then the registration and other setup won't run. That registration and other setup has to run at least once in order for the application to work correctly.

Whether or not you're running the application as an administrator, press 'T' to cause a toast to be displayed. You can then click the **Call back ToastAndCallback** button either directly from the toast notification that pops up, or from the Action Center, and your application will be launched, the coclass instantiated, and the **INotificationActivationCallback::Activate** method executed.

In-process COM server

The *ToastAndCallback* example app above functions as a local (or out-of-process) COM server. This is indicated by the [LocalServer32](#) Windows Registry key that you use to register the CLSID of its coclass. A local COM server hosts its coclass(es) inside an executable binary (an `.exe`).

Alternatively (and arguably more likely), you can choose to host your coclass(es) inside a dynamic-link library (a `.dll`). A COM server in the form of a DLL is known as an in-process COM server, and it's indicated by CLSIDs being registered by using the [InprocServer32](#) Windows Registry key.

Create a Dynamic-Link Library (DLL) project

You can begin the task of creating an in-process COM server by creating a new project in Microsoft Visual Studio. Create a **Visual C++ > Windows Desktop > Dynamic-Link Library (DLL)** project.

To add C++/WinRT support to the new project, follow the steps described in [Modify a Windows Desktop application project to add C++/WinRT support](#).

Implement the coclass, class factory, and in-proc server exports

Open `dllmain.cpp`, and add to it the code listing shown below.

If you already have a DLL that implements C++/WinRT Windows Runtime classes, then you'll already have the **DllCanUnloadNow** function shown below. If you want to add coclasses to that DLL, then you can add the **DllGetClassObject** function.

If don't have existing [Windows Runtime C++ Template Library \(WRL\)](#) code that you want to stay compatible with, then you can remove the WRL parts from the code shown.

C++/WinRT

```
// dllmain.cpp

struct MyCoclass : winrt::implements<MyCoclass, IPersist>
{
    HRESULT STDMETHODCALLTYPE GetClassID(CLSID* id) noexcept override
    {
        *id = IID_IPersist; // Doesn't matter what we return, for this
        // example.
        return S_OK;
    }
};

struct __declspec(uuid("85d6672d-0606-4389-a50a-356ce7bded09"))
    MyCoclassFactory : winrt::implements<MyCoclassFactory, IClassFactory>
{
    HRESULT STDMETHODCALLTYPE CreateInstance(IUnknown *pUnkOuter, REFIID
    riid, void **ppvObject) noexcept override
    {
        try
        {
            return winrt::make<MyCoclass>()->QueryInterface(riid,
            ppvObject);
        }
        catch (...)
        {
            return winrt::to_hresult();
        }
    }

    HRESULT STDMETHODCALLTYPE LockServer(BOOL fLock) noexcept override
    {
        // ...
        return S_OK;
    }

    // ...
};

HRESULT __stdcall DllCanUnloadNow()
{
    #ifdef _WRL_MODULE_H_

```



```

        if
        (!::Microsoft::WRL::Module<::Microsoft::WRL::InProc>::GetModule()).Terminate(
        ))
        {
            return S_FALSE;
        }
    #endif

    if (winrt::get_module_lock())
    {
        return S_FALSE;
    }

    winrt::clear_factory_cache();
    return S_OK;
}

HRESULT __stdcall DllGetClassObject(GUID const& clsid, GUID const& iid,
void** result)
{
    try
    {
        *result = nullptr;

        if (clsid == __uuidof(MyCoclassFactory))
        {
            return winrt::make<MyCoclassFactory>()->QueryInterface(iid,
result);
        }

    #ifdef _WRL_MODULE_H_
        return
        ::Microsoft::WRL::Module<::Microsoft::WRL::InProc>::GetModule().GetClassObje
ct(clsid, iid, result);
    #else
        return winrt::hresult_class_not_available().to_abi();
    #endif
    }
    catch (...)
    {
        return winrt::to_hresult();
    }
}

```

Support for weak references

Also see [Weak references in C++/WinRT](#).

C++/WinRT (specifically, the [winrt::implements](#) base struct template) implements [IWeakReferenceSource](#) for you if your type implements [IInspectable](#) (or any interface that derives from [IInspectable](#)).

This is because `IWeakReferenceSource` and `IWeakReference` are designed for Windows Runtime types. So, you can turn on weak reference support for your coclass simply by adding `winrt::Windows::Foundation::IInspectable` (or an interface that derives from `IInspectable`) to your implementation.

C++/WinRT

```
struct MyCoclass : winrt::implements<MyCoclass, IMyComInterface,  
winrt::Windows::Foundation::IInspectable>  
{  
    // ...  
};
```

Implement a COM interface that derives from another

Interface derivation is a feature of classic COM (and it happens to be absent, intentionally, from the Windows Runtime). Here's an example of what interface derivation looks like.

C++

```
IFileSystemBindData2 : public IFileSystemBindData { /* ... */ };
```

If you're writing a class that needs to implement, for example, both `IFileSystemBindData` and `IFileSystemBindData2`, then the first step in expressing that is to declare that you implement only the *derived* interface, like this.

C++/WinRT

```
// pch.h  
#pragma once  
#include <Shobjidl.h>  
...  
  
// main.cpp  
...  
struct MyFileSystemBindData :  
    implements<MyFileSystemBindData,  
    IFileSystemBindData2>  
{  
    // IFileSystemBindData  
    IFACEMETHOD(SetFindData)(const WIN32_FIND_DATA* pfd) override { /* ...  
*/ return S_OK; };  
    IFACEMETHOD(GetFindData)(WIN32_FIND_DATA* pfd) override { /* ... */  
return S_OK; };
```

```

    // IFileSystemBindData2
    IFACEMETHOD(SetFileID)(LARGE_INTEGER liFileID) override { /* ... */
return S_OK; };
    IFACEMETHOD(GetFileID)(LARGE_INTEGER* pliFileID) override { /* ... */
return S_OK; };
    IFACEMETHOD(SetJunctionCLSID)(REFCLSID clsid) override { /* ... */
return S_OK; };
    IFACEMETHOD(GetJunctionCLSID)(CLSID* pclsid) override { /* ... */ return
S_OK; };
};
...
int main()
...

```

The next step is to ensure that [QueryInterface](#) succeeds when it's called (directly or indirectly) for *IID_IFileSystemBindData* (the *base* interface) against an instance of **MyFileSystemBindData**. You do that by providing a specialization for the [winrt::is_guid_of](#) function template.

[winrt::is_guid_of](#) is variadic, and so you can provide it a list of interfaces. Here's how you'd provide a specialization so that a check for **IFileSystemBindData2** also includes a test for **IFileSystemBindData**.

C++/WinRT

```

// pch.h
...
namespace winrt
{
    template<>
    inline bool is_guid_of<IFileSystemBindData2>(guid const& id) noexcept
    {
        return is_guid_of<IFileSystemBindData2, IFileSystemBindData>(id);
    }
}

// main.cpp
...
int main()
{
    ...
    auto mfsbd{ winrt::make<MyFileSystemBindData>() };
    auto a{ mfsbd.as<IFileSystemBindData2>() }; // Would succeed even
without the **is_guid_of** specialization.
    auto b{ mfsbd.as<IFileSystemBindData>() }; // Needs the **is_guid_of**
specialization in order to succeed.
}

```

The specialization of `winrt::is_guid_of` must be identical across all files in the project, and visible at the point the interface is used by the `winrt::implements` or `winrt::delegate` template. Typically you'd put it in a common header file.

Important APIs

- [IInspectable interface](#)
- [IUnknown interface](#)
- [winrt::implements struct template](#)

Related topics

- [Author APIs with C++/WinRT](#)
- [Consume COM components with C++/WinRT](#)
- [Send a local toast notification](#)

Move to C++/WinRT from C++/CX

Article • 10/20/2022

This topic is the first in a series describing how you can port the source code in your [C++/CX](#) project to its equivalent in [C++/WinRT](#).

If your project is also using [Windows Runtime C++ Template Library \(WRL\)](#) types, then see [Move to C++/WinRT from WRL](#).

Strategies for porting

It's worth knowing that porting from C++/CX to C++/WinRT is generally straightforward, with the one exception of moving from [Parallel Patterns Library \(PPL\)](#) tasks to coroutines. The models are different. There isn't a natural one-to-one mapping from PPL tasks to coroutines, and there's no simple way to mechanically port the code that works for all cases. For help with this specific aspect of porting, and your options for interoperating between the two models, see [Asynchrony, and interop between C++/WinRT and C++/CX](#).

Development teams routinely report that once they're over the hurdle of porting their asynchronous code, the remainder of the porting work is largely mechanical.

Porting in one pass

If you're in a position to be able to port your entire project in one pass, then you'll need only this topic for the info you need (and you won't need the *interop* topics that follow this one). We recommend that you begin by creating a new project in Visual Studio using one of the C++/WinRT project templates (see [Visual Studio support for C++/WinRT](#)). Then move your source code files over into that new project, and port all of the C++/CX source code to C++/WinRT as you do so.

Alternatively, if you'd prefer to do the porting work in your existing C++/CX project, then you'll need to add C++/WinRT support to it. The steps that you follow to do that are described in [Taking a C++/CX project and adding C++/WinRT support](#). By the time you're done porting, you'll have turned what was a pure C++/CX project into a pure C++/WinRT project.

 **Note**

If you have a Windows Runtime component project, then porting in one pass is your only option. A Windows Runtime component project written in C++ must contain either all C++/CX source code, or all C++/WinRT source code. They can't coexist in this project type.

Porting a project gradually

With the exception of Windows Runtime component projects, as mentioned in the previous section, if the size or complexity of your codebase makes it necessary to port your project gradually, then you'll need a porting process in which for a time C++/CX and C++/WinRT code exists side by side in the same project. In addition to reading this topic, also see [Interop between C++/WinRT and C++/CX](#) and [Asynchrony, and interop between C++/WinRT and C++/CX](#). Those topics provide info and code examples showing how to interoperate between the two language projections.

To get a project ready for a gradual porting process, one option is to add C++/WinRT support to your C++/CX project. The steps that you follow to do that are described in [Taking a C++/CX project and adding C++/WinRT support](#). You can then port gradually from there.

Another option is to create a new project in Visual Studio using one of the C++/WinRT project templates (see [Visual Studio support for C++/WinRT](#)). And then add C++/CX support to that project. The steps that you follow to do that are described in [Taking a C++/WinRT project and adding C++/CX support](#). You can then start moving your source code over into that, and porting *some* of the C++/CX source code to C++/WinRT as you do so.

In either case, you'll interoperate (both ways) between your C++/WinRT code and any C++/CX code that you haven't yet ported.

ⓘ Note

Both C++/CX and the Windows SDK declare types in the root namespace **Windows**. A Windows type projected into C++/WinRT has the same fully-qualified name as the Windows type, but it's placed in the C++ **winrt** namespace. These distinct namespaces let you port from C++/CX to C++/WinRT at your own pace.

Porting a XAML project gradually

ⓘ Important

For a project that uses XAML, at any given time all of your XAML page types need to be either entirely C++/CX or entirely C++/WinRT. You can still mix C++/CX and C++/WinRT *outside* of XAML page types within the same project (in your models and viewmodels, and elsewhere).

For this scenario, the workflow that we recommend is to create a new C++/WinRT project and copy source code and markup over from the C++/CX project. As long as all of your XAML page types are C++/WinRT, then you can add new XAML pages with **Project > Add New Item... > Visual C++ > Blank Page (C++/WinRT)**.

Alternatively, you can use a Windows Runtime component (WRC) to factor code out of the XAML C++/CX project as you port it.

- You could create a new C++/CX WRC project, move as much C++/CX code as you can into that project, and then change the XAML project to C++/WinRT.
- Or you could create a new C++/WinRT WRC project, leave the XAML project as C++/CX, and begin porting C++/CX to C++/WinRT and moving the resulting code out of the XAML project and into the component project.
- You could also have a C++/CX component project alongside a C++/WinRT component project within the same solution, reference both of them from your application project, and gradually port from one to the other. Again, see [Interop between C++/WinRT and C++/CX](#) for more details on using the two language projections in the same project.

First steps in porting a C++/CX project to C++/WinRT

No matter what your porting strategy will be (porting in one pass, or porting gradually), your first step is to prepare your project for porting. Here's a recap of what we described in [Strategies for porting](#) in terms of the kind of project you'll be starting with, and how to set it up.

- **Porting in one pass.** Create a new project in Visual Studio using one of the C++/WinRT project templates. Move the files from your C++/CX project into that new project, and port the C++/CX source code.
- **Porting a non-XAML project gradually.** You can choose to add C++/WinRT support to your C++/CX project (see [Taking a C++/CX project and adding C++/WinRT support](#)), and port gradually. Or you can choose to create a new C++/WinRT project and add C++/CX support to that (see [Taking a C++/WinRT project and adding C++/CX support](#)), move files over, and port gradually.

- **Porting a XAML project gradually.** Create a new C++/WinRT project, move files over, and port gradually. At any given time your XAML page types must be *either* all C++/WinRT *or* all C++/CX.

The rest of this topic applies no matter which porting strategy you choose. It contains a catalog of technical details involved in porting source code from C++/CX to C++/WinRT. If you're porting gradually, then you'll likely also want to see [Interop between C++/WinRT and C++/CX](#) and [Asynchrony, and interop between C++/WinRT and C++/CX](#).

File-naming conventions

XAML markup files

File origin	C++/CX	C++/WinRT
Developer XAML files	MyPage.xaml MyPage.xaml.h MyPage.xaml.cpp	MyPage.xaml MyPage.h MyPage.cpp MyPage.idl (see below)
Generated XAML files	MyPage.xaml.g.h MyPage.xaml.g.hpp	MyPage.xaml.g.h MyPage.xaml.g.hpp MyPage.g.h

Notice that C++/WinRT removes the `.xaml` from the `*.h` and `*.cpp` file names.

C++/WinRT adds an additional developer file, the **Midl file (.idl)**. C++/CX autogenerates this file internally, adding to it every public and protected member. In C++/WinRT, you add and author the file yourself. For more details, code examples, and a walkthrough of authoring IDL, see [XAML controls; bind to a C++/WinRT property](#).

Also see [Factoring runtime classes into Midl files \(.idl\)](#)

Runtime classes

C++/CX doesn't impose restrictions on the names of your header files; it's common to put multiple runtime class definitions into a single header file, especially for small classes. But C++/WinRT requires that each runtime class has its own header file named after the class name.

C++/CX	C++/WinRT
--------	-----------

C++/CX	C++/WinRT
Common.h <pre>ref class A { ... } ref class B { ... }</pre>	Common.idl <pre>runtimeclass A { ... } runtimeclass B { ... }</pre>
	A.h <pre>namespace implements { struct A { ... }; }</pre>
	B.h <pre>namespace implements { struct B { ... }; }</pre>

Less common (but still legal) in C++/CX is to use differently-named header files for XAML custom controls. You'll need to rename these header file to match the class name.

C++/CX	C++/WinRT
A.xaml <pre><Page x:Class="LongNameForA" ...></pre>	A.xaml <pre><Page x:Class="LongNameForA" ...></pre>
A.h <pre>partial ref class LongNameForA { ... }</pre>	LongNameForA.h <pre>namespace implements { struct LongNameForA { ... }; }</pre>

Header file requirements

C++/CX doesn't require you to include any special header files, because it internally autogenerates header files from `.winmd` files. It's common in C++/CX to use `using` directives for namespaces that you consume by name.

C++/CX
<pre>using namespace Windows::Media::Playback; String^ NameOfFirstVideoTrack(MediaPlaybackItem^ item) { return item->VideoTracks->GetAt(0)->Name; }</pre>

The `using namespace Windows::Media::Playback` directive lets us write `MediaPlaybackItem` without a namespace prefix. We also touched the

`Windows.Media.Core` namespace, because `item->VideoTracks->GetAt(0)` returns a **Windows.Media.Core.VideoTrack**. But we didn't have to type the name **VideoTrack** anywhere, so we didn't need a `using Windows.Media.Core` directive.

But C++/WinRT requires you to include a header file corresponding to each namespace that you consume, even if you don't name it.

C++/WinRT

```
#include <winrt/Windows.Media.Playback.h>
#include <winrt/Windows.Media.Core.h> // !!This is important!!

using namespace winrt;
using namespace Windows::Media::Playback;

winrt::hstring NameOfFirstVideoTrack(MediaPlaybackItem const& item)
{
    return item.VideoTracks().GetAt(0).Name();
}
```

On the other hand, even though the **MediaPlaybackItem.AudioTracksChanged** event is of type **TypedEventHandler<MediaPlaybackItem, Windows.Foundation.Collections.IVectorChangedEventArgs>**, we don't need to include `winrt/Windows.Foundation.Collections.h` because we didn't use that event.

C++/WinRT also requires you to include header files for namespaces that are consumed by XAML markup.

XAML

```
<!-- MainPage.xaml -->
<Rectangle Height="400"/>
```

Using the **Rectangle** class means that you have to add this include.

C++/WinRT

```
// MainPage.h
#include <winrt/Windows.UI.Xaml.Shapes.h>
```

If you forget a header file, then everything will compile okay but you'll get linker errors because the `consume_` classes are missing.

Parameter-passing

When writing C++/CX source code, you pass C++/CX types as function parameters as hat (^) references.

C++/CX

```
void LogPresenceRecord(PresenceRecord^ record);
```

In C++/WinRT, for synchronous functions, you should use `const&` parameters by default. That will avoid copies and interlocked overhead. But your coroutines should use pass-by-value to ensure that they capture by value and avoid lifetime issues (for more details, see [Concurrency and asynchronous operations with C++/WinRT](#)).

C++/WinRT

```
void LogPresenceRecord(PresenceRecord const& record);  
IASyncAction LogPresenceRecordAsync(PresenceRecord const record);
```

A C++/WinRT object is fundamentally a value that holds an interface pointer to the backing Windows Runtime object. When you copy a C++/WinRT object, the compiler copies the encapsulated interface pointer, incrementing its reference count. Eventual destruction of the copy involves decrementing the reference count. So, only incur the overhead of a copy when necessary.

Variable and field references

When writing C++/CX source code, you use hat (^) variables to reference Windows Runtime objects, and the arrow (->) operator to dereference a hat variable.

C++/CX

```
IVectorView<User^>^ userList = User::Users;  
  
if (userList != nullptr)  
{  
    for (UINT32 iUser = 0; iUser < userList->Size; ++iUser)  
        ...  
}
```

When porting to the equivalent C++/WinRT code, you can get a long way by removing the hats, and changing the arrow operator (->) to the dot operator (.). C++/WinRT projected types are values, and not pointers.

C++/WinRT

```

IVectorView<User> userList = User::Users();

if (userList != nullptr)
{
    for (UINT32 iUser = 0; iUser < userList.Size(); ++iUser)
        ...

```

The default constructor for a C++/CX hat reference initializes it to null. Here's a C++/CX code example in which we create a variable/field of the correct type, but one that's uninitialized. In other words, it doesn't initially refer to a **TextBlock**; we intend to assign a reference later.

C++/CX

```

TextBlock^ textBlock;

class MyClass
{
    TextBlock^ textBlock;
};

```

For the equivalent in C++/WinRT, see [Delayed initialization](#).

Properties

The C++/CX language extensions include the concept of properties. When writing C++/CX source code, you can access a property as if it were a field. Standard C++ does not have the concept of a property so, in C++/WinRT, you call get and set functions.

In the examples that follow, **XboxUserId**, **UserState**, **PresenceDeviceRecords**, and **Size** are all properties.

Retrieving a value from a property

Here's how you get a property value in C++/CX.

C++/CX

```

void Sample::LogPresenceRecord(PresenceRecord^ record)
{
    auto id = record->XboxUserId;
    auto state = record->UserState;
    auto size = record->PresenceDeviceRecords->Size;
}

```

The equivalent C++/WinRT source code calls a function with the same name as the property, but with no parameters.

C++/WinRT

```
void Sample::LogPresenceRecord(PresenceRecord const& record)
{
    auto id = record.XboxUserId();
    auto state = record.UserState();
    auto size = record.PresenceDeviceRecords().Size();
}
```

Note that the **PresenceDeviceRecords** function returns a Windows Runtime object that itself has a **Size** function. As the returned object is also a C++/WinRT projected type, we dereference using the dot operator to call **Size**.

Setting a property to a new value

Setting a property to a new value follows a similar pattern. First, in C++/CX.

C++/CX

```
record->UserState = newValue;
```

To do the equivalent in C++/WinRT, you call a function with the same name as the property, and pass an argument.

C++/WinRT

```
record.UserState(newValue);
```

Creating an instance of a class

You work with a C++/CX object via a handle to it, commonly known as a hat (^) reference. You create a new object via the `ref new` keyword, which in turn calls **RoActivateInstance** to activate a new instance of the runtime class.

C++/CX

```
using namespace Windows::Storage::Streams;

class Sample
{
private:
```

```
    Buffer^ m_gamerPicBuffer = ref new Buffer(MAX_IMAGE_SIZE);  
};
```

A C++/WinRT object is a value; so you can allocate it on the stack, or as a field of an object. You *never* use `ref new` (nor `new`) to allocate a C++/WinRT object. Behind the scenes, **RoActivateInstance** is still being called.

C++/WinRT

```
using namespace winrt::Windows::Storage::Streams;  
  
struct Sample  
{  
private:  
    Buffer m_gamerPicBuffer{ MAX_IMAGE_SIZE };  
};
```

If a resource is expensive to initialize, then it's common to delay initialization of it until it's actually needed. As already mentioned, the default constructor for a C++/CX hat reference initializes it to null.

C++/CX

```
using namespace Windows::Storage::Streams;  
  
class Sample  
{  
public:  
    void DelayedInit()  
    {  
        // Allocate the actual buffer.  
        m_gamerPicBuffer = ref new Buffer(MAX_IMAGE_SIZE);  
    }  
  
private:  
    Buffer^ m_gamerPicBuffer;  
};
```

The same code ported to C++/WinRT. Note the use of the `std::nullptr_t` constructor. For more info about that constructor, see [Delayed initialization](#).

C++/WinRT

```
using namespace winrt::Windows::Storage::Streams;  
  
struct Sample  
{  
    void DelayedInit()
```

```

{
    // Allocate the actual buffer.
    m_gamerPicBuffer = Buffer(MAX_IMAGE_SIZE);
}

private:
    Buffer m_gamerPicBuffer{ nullptr };
};

```

How the default constructor affects collections

C++ collection types use the default constructor, which can result in unintended object construction.

Scenario	C++/CX	C++/WinRT (incorrect)	C++/WinRT (correct)
Local variable, initially empty	<code>TextBox^ textBox;</code>	<code>TextBox textBox; //</code> <code>Creates a TextBox!</code>	<code>TextBox textBox{ nullptr }</code> <code>};</code>
Member variable, initially empty	<code>class C {</code> <code> TextBox^ textBox;</code> <code>};</code>	<code>class C {</code> <code> TextBox textBox; //</code> <code>Creates a TextBox!</code> <code>};</code>	<code>class C {</code> <code> TextBox textbox{ nullptr }</code> <code>};</code> <code>};</code>
Global variable, initially empty	<code>TextBox^ g_textBox;</code>	<code>TextBox g_textBox;</code> <code>// Creates a</code> <code>TextBox!</code>	<code>TextBox g_textBox{</code> <code>nullptr };</code>
Vector of empty references	<code>std::vector<TextBox^></code> <code>boxes(10);</code>	<code>// Creates 10</code> <code>TextBox objects!</code> <code>std::vector<TextBox></code> <code>boxes(10);</code>	<code>std::vector<TextBox></code> <code>boxes(10, nullptr);</code>
Set a value in a map	<code>std::map<int,</code> <code>TextBox^> boxes;</code> <code>boxes[2] = value;</code>	<code>std::map<int,</code> <code>TextBox> boxes;</code> <code>// Creates a TextBox</code> <code>at 2,</code> <code>// then overwrites</code> <code>it!</code> <code>boxes[2] = value;</code>	<code>std::map<int, TextBox></code> <code>boxes;</code> <code>boxes.insert_or_assign(2,</code> <code>value);</code>
Array of empty references	<code>TextBox^ boxes[2];</code>	<code>// Creates 2 TextBox</code> <code>objects!</code> <code>TextBox boxes[2];</code>	<code>TextBox boxes[2] = {</code> <code>nullptr, nullptr };</code>

Scenario	C++/CX	C++/WinRT (incorrect)	C++/WinRT (correct)
Pair	<code>std::pair<TextBox^, String^> p;</code>	<code>// Creates a TextBox! std::pair<TextBox, String> p;</code>	<code>std::pair<TextBox, String> p{ nullptr, nullptr };</code>

More about collections of empty references

Whenever you have a `Platform::Array^` (see [Port Platform::Array^](#)) in C++/CX, you have the choice to port that to a `std::vector` in C++/WinRT (in fact, any contiguous container) rather than leave it as an array. There are advantages to choosing `std::vector`.

For example, while there is shorthand for creating a fixed-sized vector of empty references (see table above), there's no such shorthand for creating an *array* of empty references. You have to repeat `nullptr` for each element in an array. If you have too few, then the extras will be default-constructed.

For a vector, you can fill it with empty references at initialization (as in the table above), or you can fill it with empty references post-initialization with code such as this.

C++/WinRT

```
std::vector<TextBox> boxes(10); // 10 default-constructed TextBoxes.
boxes.resize(10, nullptr); // 10 empty references.
```

More about the std::map example

The `[]` subscript operator for `std::map` behaves like this.

- If the key is found in the map, return a reference to the existing value (which you can overwrite).
- If the key isn't found in the map, then create a new entry in the map consisting of the key (moved, if movable) and a *default-constructed value*, and return a reference to the value (which you can then overwrite).

In other words, the `[]` operator always creates an entry in the map. This is different from C#, Java, and JavaScript.

Converting from a base runtime class to a derived one

It's common to have a reference-to-base that you know refers to an object of a derived type. In C++/CX, you use `dynamic_cast` to cast the reference-to-base into a reference-to-derived. The `dynamic_cast` is really just a hidden call to [QueryInterface](#). Here's a typical example—you're handling a dependency property changed event, and you want to cast from **DependencyObject** back to the actual type that owns the dependency property.

C++/CX

```
void BgLabelControl::OnLabelChanged(Windows::UI::Xaml::DependencyObject^ d,
Windows::UI::Xaml::DependencyPropertyChangedEventArgs^ e)
{
    BgLabelControl^ theControl{ dynamic_cast<BgLabelControl^>(d) };

    if(theControl != nullptr)
    {
        // succeeded ...
    }
}
```

The equivalent C++/WinRT code replaces the `dynamic_cast` with a call to the [IUnknown::try_as](#) function, which encapsulates **QueryInterface**. You also have the option to call [IUnknown::as](#), instead, which throws an exception if querying for the required interface (the default interface of the type you're requesting) is not returned. Here's a C++/WinRT code example.

C++/WinRT

```
void BgLabelControl::OnLabelChanged(Windows::UI::Xaml::DependencyObject
const& d, Windows::UI::Xaml::DependencyPropertyChangedEventArgs const& e)
{
    if (BgLabelControlApp::BgLabelControl theControl{
d.try_as<BgLabelControlApp::BgLabelControl>() })
    {
        // succeeded ...
    }

    try
    {
        BgLabelControlApp::BgLabelControl theControl{
d.as<BgLabelControlApp::BgLabelControl>() };
        // succeeded ...
    }
    catch (winrt::hresult_no_interface const&)
```

```

    {
        // failed ...
    }
}

```

Derived classes

In order to derive from a runtime class, the base class must be *composable*. C++/CX doesn't require that you take any special steps to make your classes composable, but C++/WinRT does. You use the [unsealed keyword](#) to indicate that you want your class to be usable as a base class.

idl

```

unsealed runtimeclass BasePage : Windows.UI.Xaml.Controls.Page
{
    ...
}
runtimeclass DerivedPage : BasePage
{
    ...
}

```

In your implementation header class, you must include the base class header file before you include the autogenerated header for the derived class. Otherwise you'll get errors such as "Illegal use of this type as an expression".

C++/WinRT

```

// DerivedPage.h
#include "BasePage.h"           // This comes first.
#include "DerivedPage.g.h"      // Otherwise this header file will produce an
error.

namespace winrt::MyNamespace::implementation
{
    struct DerivedPage : DerivedPageT<DerivedPage>
    {
        ...
    }
}

```

Event-handling with a delegate

Here's a typical example of handling an event in C++/CX, using a lambda function as a delegate in this case.

C++/CX

```
auto token = myButton->Click += ref new RoutedEventHandler( [= ]
(Platform::Object^ sender, RoutedEventArgs^ args)
{
    // Handle the event.
    // Note: locals are captured by value, not reference, since this handler
    is delayed.
});
```

This is the equivalent in C++/WinRT.

C++/WinRT

```
auto token = myButton().Click( [= ](IInspectable const& sender,
RoutedEventArgs const& args)
{
    // Handle the event.
    // Note: locals are captured by value, not reference, since this handler
    is delayed.
});
```

Instead of a lambda function, you can choose to implement your delegate as a free function, or as a pointer-to-member-function. For more info, see [Handle events by using delegates in C++/WinRT](#).

If you're porting from a C++/CX codebase where events and delegates are used internally (not across binaries), then [winrt::delegate](#) will help you to replicate that pattern in C++/WinRT. Also see [Parameterized delegates, simple signals, and callbacks within a project](#).

Revoking a delegate

In C++/CX you use the `-=` operator to revoke a prior event registration.

C++/CX

```
myButton->Click -= token;
```

This is the equivalent in C++/WinRT.

C++/WinRT

```
myButton().Click(token);
```

For more info and options, see [Revoke a registered delegate](#).

Boxing and unboxing

C++/CX automatically boxes scalars into objects. C++/WinRT requires you to call the [winrt::box_value](#) function explicitly. Both languages require you to unbox explicitly. See [Boxing and unboxing with C++/WinRT](#).

In the tables that follows, we'll use these definitions.

C++/CX	C++/WinRT
<code>int i;</code>	<code>int i;</code>
<code>String^ s;</code>	<code>winrt::hstring s;</code>
<code>Object^ o;</code>	<code>IInspectable o;</code>

Operation	C++/CX	C++/WinRT
Boxing	<code>o = 1;</code> <code>o = "string";</code>	<code>o = box_value(1);</code> <code>o = box_value(L"string");</code>
Unboxing	<code>i = (int)o;</code> <code>s = (String^)o;</code>	<code>i = unbox_value<int>(o);</code> <code>s = unbox_value<winrt::hstring>(o);</code>

C++/CX and C# raise exceptions if you try to unbox a null pointer to a value type. C++/WinRT considers this a programming error, and it crashes. In C++/WinRT, use the [winrt::unbox_value_or](#) function if you want to handle the case where the object is not of the type that you thought it was.

Scenario	C++/CX	C++/WinRT
Unbox a known integer	<code>i = (int)o;</code>	<code>i = unbox_value<int>(o);</code>
If o is null	<code>Platform::NullReferenceException</code>	Crash
If o is not a boxed int	<code>Platform::InvalidCastException</code>	Crash
Unbox int, use fallback if null; crash if anything else	<code>i = o ? (int)o : fallback;</code>	<code>i = o ? unbox_value<int>(o) : fallback;</code>

Scenario	C++/CX	C++/WinRT
Unbox int if possible; use fallback for anything else	<pre>auto box = dynamic_cast<IBox<int>^>(o); i = box ? box->Value : fallback;</pre>	<pre>i = unbox_value_or<int> (o, fallback);</pre>

Boxing and unboxing a string

A string is in some ways a value type, and in other ways a reference type. C++/CX and C++/WinRT treat strings differently.

The ABI type **HSTRING** is a pointer to a reference-counted string. But it doesn't derive from **IInspectable**, so it's not technically an *object*. Furthermore, a null **HSTRING** represents the empty string. Boxing of things not derived from **IInspectable** is done by wrapping them inside an **IReference<T>**, and the Windows Runtime provides a standard implementation in the form of the **PropertyValue** object (custom types are reported as **PropertyType::OtherType**).

C++/CX represents a Windows Runtime string as a reference type; while C++/WinRT projects a string as a value type. This means that a boxed null string can have different representations depending how you got there.

Furthermore, C++/CX allows you to dereference a null **String^**, in which case it behaves like the string `""`.

Behavior	C++/CX	C++/WinRT
Declarations	<pre>Object^ o; String^ s;</pre>	<pre>IInspectable o; hstring s;</pre>
String type category	Reference type	Value type
null HSTRING projects as	<code>(String^)nullptr</code>	<code>hstring{}</code>
Are null and <code>""</code> identical?	Yes	Yes
Validity of null	<pre>s = nullptr; s->Length == 0 (valid)</pre>	<pre>s = hstring{}; s.size() == 0 (valid)</pre>
If you assign null string to object	<pre>o = (String^)nullptr; o == nullptr</pre>	<pre>o = box_value(hstring{}); o != nullptr</pre>
If you assign <code>""</code> to object	<pre>o = ""; o == nullptr</pre>	<pre>o = box_value(hstring{L""}); o != nullptr</pre>

Basic boxing and unboxing.

Operation	C++/CX	C++/WinRT
Box a string	<code>o = s;</code> Empty string becomes nullptr.	<code>o = box_value(s);</code> Empty string becomes non-null object.
Unbox a known string	<code>s = (String^)o;</code> Null object becomes empty string. InvalidCastException if not a string.	<code>s = unbox_value<hstring>(o);</code> Null object crashes. Crash if not a string.
Unbox a possible string	<code>s = dynamic_cast<String^>(o);</code> Null object or non-string becomes empty string.	<code>s = unbox_value_or<hstring>(o, fallback);</code> Null or non-string becomes fallback. Empty string preserved.

Concurrency and asynchronous operations

The Parallel Patterns Library (PPL) ([concurrency::task](#), for example) was updated to support C++/CX hat references.

For C++/WinRT, you should use coroutines and `co_await` instead. For more info, and code examples, see [Concurrency and asynchronous operations with C++/WinRT](#).

Consuming objects from XAML markup

In a C++/CX project, you can consume private members and named elements from XAML markup. But in C++/WinRT, all entities consumed by using the XAML `{x:Bind}` [markup extension](#) must be exposed publicly in IDL.

Also, binding to a Boolean displays `true` or `false` in C++/CX, but it shows `Windows.Foundation.IReference`1<Boolean>` in C++/WinRT.

For more info, and code examples, see [Consuming objects from markup](#).

Mapping C++/CX Platform types to C++/WinRT types

C++/CX provides several data types in the **Platform** namespace. These types are not standard C++, so you can only use them when you enable Windows Runtime language extensions (Visual Studio project property **C/C++ > General > Consume Windows Runtime Extension > Yes (/ZW)**). The table below helps you port from **Platform** types to

their equivalents in C++/WinRT. Once you've done that, since C++/WinRT is standard C++, you can turn off the `/ZW` option.

C++/CX	C++/WinRT
<code>Platform::Agile^</code>	winrt::agile_ref
<code>Platform::Array^</code>	See Port Platform::Array^
<code>Platform::Exception^</code>	winrt::hresult_error
<code>Platform::InvalidArgumentException^</code>	winrt::hresult_invalid_argument
<code>Platform::Object^</code>	<code>winrt::Windows::Foundation::IInspectable</code>
<code>Platform::String^</code>	winrt::hstring

Port `Platform::Agile^` to `winrt::agile_ref`

The `Platform::Agile^` type in C++/CX represents a Windows Runtime class that can be accessed from any thread. The C++/WinRT equivalent is [winrt::agile_ref](#).

In C++/CX.

C++/CX

```
Platform::Agile<Windows::UI::Core::CoreWindow> m_window;
```

In C++/WinRT.

C++/WinRT

```
winrt::agile_ref<Windows::UI::Core::CoreWindow> m_window;
```

Port `Platform::Array^`

In cases where C++/CX requires you to use an array, C++/WinRT allows you to use any contiguous container. See [How the default constructor affects collections](#) for a reason why `std::vector` is a good choice.

So, whenever you have a `Platform::Array^` in C++/CX, your porting options include using an initializer list, a `std::array`, or a `std::vector`. For more info, and code examples, see [Standard initializer lists](#) and [Standard arrays and vectors](#).

Port Platform::Exception^ to winrt::hresult_error

The **Platform::Exception^** type is produced in C++/CX when a Windows Runtime API returns a non S_OK HRESULT. The C++/WinRT equivalent is [winrt::hresult_error](#).

To port to C++/WinRT, change all code that uses **Platform::Exception^** to use **winrt::hresult_error**.

In C++/CX.

C++/CX

```
catch (Platform::Exception^ ex)
```

In C++/WinRT.

C++/WinRT

```
catch (winrt::hresult_error const& ex)
```

C++/WinRT provides these exception classes.

Exception type	Base class	HRESULT
winrt::hresult_error		call hresult_error::to_abi
winrt::hresult_access_denied	winrt::hresult_error	E_ACCESSDENIED
winrt::hresult_canceled	winrt::hresult_error	ERROR_CANCELLED
winrt::hresult_changed_state	winrt::hresult_error	E_CHANGED_STATE
winrt::hresult_class_not_available	winrt::hresult_error	CLASS_E_CLASSNOTAVAILABLE
winrt::hresult_illegal_delegate_assignment	winrt::hresult_error	E_ILLEGAL_DELEGATE_ASSIGNMENT
winrt::hresult_illegal_method_call	winrt::hresult_error	E_ILLEGAL_METHOD_CALL
winrt::hresult_illegal_state_change	winrt::hresult_error	E_ILLEGAL_STATE_CHANGE
winrt::hresult_invalid_argument	winrt::hresult_error	E_INVALIDARG
winrt::hresult_no_interface	winrt::hresult_error	E_NOINTERFACE
winrt::hresult_not_implemented	winrt::hresult_error	E_NOTIMPL
winrt::hresult_out_of_bounds	winrt::hresult_error	E_BOUNDS
winrt::hresult_wrong_thread	winrt::hresult_error	RPC_E_WRONG_THREAD

Note that each class (via the `hresult_error` base class) provides a [to_abi](#) function, which returns the HRESULT of the error, and a [message](#) function, which returns the string representation of that HRESULT.

Here's an example of throwing an exception in C++/CX.

C++/CX

```
throw ref new Platform::InvalidArgumentException(L"A valid User is required");
```

And the equivalent in C++/WinRT.

C++/WinRT

```
throw winrt::hresult_invalid_argument{ L"A valid User is required" };
```

Port Platform::Object^ to winrt::Windows::Foundation::IInspectable

Like all C++/WinRT types, `winrt::Windows::Foundation::IInspectable` is a value type. Here's how you initialize a variable of that type to null.

C++/WinRT

```
winrt::Windows::Foundation::IInspectable var{ nullptr };
```

Port Platform::String^ to winrt::hstring

`Platform::String^` is equivalent to the Windows Runtime HSTRING ABI type. For C++/WinRT, the equivalent is [winrt::hstring](#). But with C++/WinRT, you can call Windows Runtime APIs using C++ Standard Library wide string types such as `std::wstring`, and/or wide string literals. For more details, and code examples, see [String handling in C++/WinRT](#).

With C++/CX, you can access the [Platform::String::Data](#) property to retrieve the string as a C-style `const wchar_t*` array (for example, to pass it to `std::wcout`).

C++/CX

```
auto var{ titleRecord->TitleName->Data() };
```

To do the same with C++/WinRT, you can use the [hstring::c_str](#) function to get a null-terminated C-style string version, just as you can from `std::wstring`.

C++/WinRT

```
auto var{ titleRecord.TitleName().c_str() };
```

When it comes to implementing APIs that take or return strings, you typically change any C++/CX code that uses `Platform::String^` to use `winrt::hstring` instead.

Here's an example of a C++/CX API that takes a string.

C++/CX

```
void LogWrapLine(Platform::String^ str);
```

For C++/WinRT you could declare that API in [MIDL 3.0](#) like this.

idl

```
// LogType.idl
void LogWrapLine(String str);
```

The C++/WinRT toolchain will then generate source code for you that looks like this.

C++/WinRT

```
void LogWrapLine(winrt::hstring const& str);
```

ToString()

C++/CX types provide the [Object::ToString](#) method.

C++/CX

```
int i{ 2 };
auto s{ i.ToString() }; // s is a Platform::String^ with value L"2".
```

C++/WinRT doesn't directly provide this facility, but you can turn to alternatives.

C++/WinRT

```
int i{ 2 };
auto s{ std::to_wstring(i) }; // s is a std::wstring with value L"2".
```

C++/WinRT also supports `winrt::to_hstring` for a limited number of types. You'll need to add overloads for any additional types you want to stringify.

Language	Stringify int	Stringify enum
C++/CX	<pre>String^ result = "hello, " + intValue.ToString();</pre>	<pre>String^ result = "status: " + status.ToString();</pre>
C++/WinRT	<pre>hstring result = L"hello, " + to_hstring(intValue);</pre>	<pre>// must define overload (see below) hstring result = L"status: " + to_hstring(status);</pre>

In the case of stringifying an enum, you will need to provide the implementation of `winrt::to_hstring`.

C++/WinRT

```
namespace winrt
{
    hstring to_hstring(StatusEnum status)
    {
        switch (status)
        {
            case StatusEnum::Success: return L"Success";
            case StatusEnum::AccessDenied: return L"AccessDenied";
            case StatusEnum::DisabledByPolicy: return L"DisabledByPolicy";
            default: return to_hstring(static_cast<int>(status));
        }
    }
}
```

These stringifications are often consumed implicitly by data binding.

XAML

```
<TextBlock>  
You have <Run Text="{Binding FlowerCount}"/> flowers.  
</TextBlock>  
<TextBlock>  
Most recent status is <Run Text="{x:Bind LatestOperation.Status}"/>.  
</TextBlock>
```

These bindings will perform `winrt::to_hstring` of the bound property. In the case of the second example (the `StatusEnum`), you must provide your own overload of `winrt::to_hstring`, otherwise you'll get a compiler error.

String-building

C++/CX and C++/WinRT defer to the standard `std::wstringstream` for string building.

Operation	C++/CX	C++/WinRT
Append string, preserving nulls	<code>stream.print(s->Data(), s->Length);</code>	<code>stream << std::wstring_view{ s };</code>
Append string, stop on first null	<code>stream << s->Data();</code>	<code>stream << s.c_str();</code>
Extract result	<code>ws = stream.str();</code>	<code>ws = stream.str();</code>

More examples

In the examples below, `ws` is a variable of type `std::wstring`. Also, while C++/CX can construct a `Platform::String` from an 8-bit string, C++/WinRT doesn't do that.

Operation	C++/CX	C++/WinRT
Construct string from literal	<code>String^ s = "hello";</code> <code>String^ s = L"hello";</code>	<code>// winrt::hstring s{ "hello" };</code> <code>// Doesn't compile</code> <code>winrt::hstring s{ L"hello" };</code>
Convert from <code>std::wstring</code> , preserving nulls	<code>String^ s = ref new</code> <code>String(ws.c_str(),</code> <code>(uint32_t)ws.size());</code>	<code>winrt::hstring s{ ws };</code> <code>s = winrt::hstring(ws);</code> <code>// s = ws; // Doesn't compile</code>
Convert from <code>std::wstring</code> , stop on first null	<code>String^ s = ref new</code> <code>String(ws.c_str());</code>	<code>winrt::hstring s{ ws.c_str() };</code> <code>s = winrt::hstring(ws.c_str());</code> <code>// s = ws.c_str(); // Doesn't compile</code>
Convert to <code>std::wstring</code> , preserving nulls	<code>std::wstring ws{ s->Data(),</code> <code>s->Length };</code> <code>ws = std::wstring(s->Data(),</code> <code>s->Length);</code>	<code>std::wstring ws{ s };</code> <code>ws = s;</code>
Convert to <code>std::wstring</code> , stop on first null	<code>std::wstring ws{ s->Data()</code> <code>};</code> <code>ws = s->Data();</code>	<code>std::wstring ws{ s.c_str() };</code> <code>ws = s.c_str();</code>
Pass literal to method	<code>Method("hello");</code> <code>Method(L"hello");</code>	<code>// Method("hello"); // Doesn't compile</code> <code>Method(L"hello");</code>

Operation	C++/CX	C++/WinRT
Pass <code>std::wstring</code> to method	<pre>Method(ref new String(ws.c_str(), (uint32_t)ws.size()); // Stops on first null</pre>	<pre>Method(ws); // param::winrt::hstring accepts std::wstring_view</pre>

Important APIs

- [winrt::delegate struct template](#)
- [winrt::hresult_error struct](#)
- [winrt::hstring struct](#)
- [winrt namespace](#)

Related topics

- [C++/CX](#)
- [Author events in C++/WinRT](#)
- [Concurrency and asynchronous operations with C++/WinRT](#)
- [Consume APIs with C++/WinRT](#)
- [Handle events by using delegates in C++/WinRT](#)
- [Interop between C++/WinRT and C++/CX](#)
- [Asynchrony, and interop between C++/WinRT and C++/CX](#)
- [Microsoft Interface Definition Language 3.0 reference](#)
- [Move to C++/WinRT from WRL](#)
- [String handling in C++/WinRT](#)

Interop between C++/WinRT and C++/CX

Article • 10/20/2022

Before reading this topic, you'll need the info in the topic [Move to C++/WinRT from C++/CX](#). That topic introduces two main strategy options for porting your C++/CX project to C++/WinRT.

- Port the entire project in one pass. The simplest option for a project that's not too large. If you have a Windows Runtime component project, then this strategy is your only option.
- Port the project gradually (the size or complexity of your codebase might make this necessary). But this strategy calls for you to follow a porting process in which for a time C++/CX and C++/WinRT code exists side by side in the same project. For a XAML project, at any given time, your XAML page types must be *either* all C++/WinRT *or* all C++/CX.

This interop topic is relevant for that *second* strategy—for cases when you need to port your project gradually. This topic shows you various forms of pairs of helper functions that you can use to convert a C++/CX object (and other types) into a C++/WinRT object (and vice versa) within the same project.

These helper functions will be very useful as you port your code gradually from C++/CX to C++/WinRT. Or you might just choose to use both the C++/WinRT and C++/CX language projections in the same project, whether you're porting or not, and use these helper functions to interoperate between the two.

After reading this topic, for info and code examples showing how to support PPL tasks and coroutines side by side in the same project (for example, calling coroutines from task chains), see the more advanced topic [Asynchrony, and interop between C++/WinRT and C++/CX](#).

The `from_cx` and `to_cx` functions

Here's a source code listing of a header file named `interop_helpers.h`, containing various conversion helper functions. As you gradually port your project, there will be parts still in C++/CX, and parts that you've ported to C++/WinRT. You can use these helper functions to convert objects (and other types) to and from C++/CX and C++/WinRT in your project at the boundary points between those two parts.

The sections that follow the code listing explain the helper functions, and how to create and use the header file in your project.

C++/WinRT

```
// interop_helpers.h
#pragma once

template <typename T>
T from_cx(Platform::Object^ from)
{
    T to{ nullptr };

    if (from != nullptr)
    {
        winrt::check_hresult(reinterpret_cast<::IUnknown*>(from)
            ->QueryInterface(winrt::guid_of<T>(), winrt::put_abi(to)));
    }

    return to;
}

template <typename T>
T^ to_cx(winrt::Windows::Foundation::IUnknown const& from)
{
    return safe_cast<T^>(reinterpret_cast<Platform::Object^>
        (winrt::get_abi(from)));
}

inline winrt::hstring from_cx(Platform::String^ const& from)
{
    return reinterpret_cast<winrt::hstring&>(const_cast<Platform::String^>
        (from));
}

inline Platform::String^ to_cx(winrt::hstring const& from)
{
    return reinterpret_cast<Platform::String^>(const_cast<winrt::hstring&>
        (from));
}

inline winrt::guid from_cx(Platform::Guid const& from)
{
    return reinterpret_cast<winrt::guid&>(const_cast<Platform::Guid&>
        (from));
}

inline Platform::Guid to_cx(winrt::guid const& from)
{
    return reinterpret_cast<Platform::Guid&>(const_cast<winrt::guid&>
        (from));
}
```

The from_cx function

The `from_cx` helper function converts a C++/CX object to an equivalent C++/WinRT object. The function casts a C++/CX object to its underlying `IUnknown` interface pointer. It then calls `QueryInterface` on that pointer to query for the default interface of the C++/WinRT object. `QueryInterface` is the Windows Runtime application binary interface (ABI) equivalent of the C++/CX `safe_cast` extension. And, the `wintr::put_abi` function retrieves the address of a C++/WinRT object's underlying `IUnknown` interface pointer so that it can be set to another value.

The to_cx function

The `to_cx` helper function converts a C++/WinRT object to an equivalent C++/CX object. The `wintr::get_abi` function retrieves a pointer to a C++/WinRT object's underlying `IUnknown` interface. The function casts that pointer to a C++/CX object before using the C++/CX `safe_cast` extension to query for the requested C++/CX type.

The `interop_helpers.h` header file

To use the helper functions in your project, follow these steps.

- Add a new **Header File (.h)** item to your project, and name it `interop_helpers.h`.
- Replace the contents of `interop_helpers.h` with the code listing above.
- Add these includes to `pch.h`.

C++/WinRT

```
// pch.h
...
#include <unknwn.h>
// Include C++/WinRT projected Windows API headers here.
...
#include <interop_helpers.h>
```

Taking a C++/CX project and adding C++/WinRT support

This section describes what to do if you've decided to take your existing C++/CX project, add C++/WinRT support to it, and do your porting work there. Also see [Visual Studio support for C++/WinRT](#).

To mix C++/CX and C++/WinRT in a C++/CX project—including using the **from_cx** and **to_cx** helper functions in the project—you'll need to manually add C++/WinRT support to the project.

First, open your C++/CX project in Visual Studio and confirm that project property **General > Target Platform Version** is set to 10.0.17134.0 (Windows 10, version 1803) or greater.

Install the C++/WinRT NuGet package

The [Microsoft.Windows.CppWinRT NuGet package](#) provides C++/WinRT build support (MSBuild properties and targets). To install it, click the menu item **Project > Manage NuGet Packages... > Browse**, type or paste **Microsoft.Windows.CppWinRT** in the search box, select the item in search results, and then click **Install** to install the package for that project.

❗ Important

Installing the C++/WinRT NuGet package causes support for C++/CX to be turned off in the project. If you're going to port in one pass, then it's a good idea to leave that support turned off so that build messages will help you find (and port) all of your dependencies on C++/CX (eventually turning what was a pure C++/CX project into a pure C++/WinRT project). But see the next section for info about turning it back on.

Turn C++/CX support back on

If you're porting in one pass, then you don't need to do this. But if you need to port gradually, then at this point you'll need to turn C++/CX support back on in your project. In project properties, **C/C++ > General > Consume Windows Runtime Extension > Yes (/ZW)**.

Alternatively (or, for a XAML project, in addition), you can add C++/CX support by using the C++/WinRT project property page in Visual Studio. In project properties, **Common Properties > C++/WinRT > Project Language > C++/CX**. Doing that will add the following property to your `.vcxproj` file.

XML

```
<PropertyGroup Label="Globals">
  <CppWinRTProjectLanguage>C++/CX</CppWinRTProjectLanguage>
</PropertyGroup>
```

❗ Important

Whenever you need to build to process the contents of a **Midl File (.idl)** into stub files, you'll need to change **Project Language** back to **C++/WinRT**. After the build has generated those stubs, change **Project Language** back to **C++/CX**.

For a list of similar customization options (which fine-tune the behavior of the `cppwinrt.exe` tool), see the Microsoft.Windows.CppWinRT NuGet package [readme](#).

Include C++/WinRT header files

The least you should do is, in your precompiled header file (usually `pch.h`), include `winrt/base.h` as shown below.

C++/WinRT

```
// pch.h
...
#include <winrt/base.h>
...
```

But you'll almost certainly need the types in the `winrt::Windows::Foundation` namespace. And you might already know of other namespaces that you'll need. So include the C++/WinRT projected Windows API headers that correspond to those namespaces like this (you don't need to explicitly include `winrt/base.h` now because it will be included automatically for you).

C++/WinRT

```
// pch.h
...
#include <winrt/Windows.Foundation.h>
// Include any other C++/WinRT projected Windows API headers here.
...
```

Also see the code example in the following section (*Taking a C++/WinRT project and adding C++/CX support*) for a technique using the namespace aliases `namespace cx` and `namespace winrt`. That technique lets you deal with otherwise potential namespace collisions between the C++/WinRT projection and the C++/CX projection.

Add `interop_helpers.h` to the project

You'll now be able to add the `from_cx` and `to_cx` functions to your C++/CX project. For instructions on doing that, see the [from_cx and to_cx functions](#) section above.

Taking a C++/WinRT project and adding C++/CX support

This section describes what to do if you've decided to create a new C++/WinRT project, and do your porting work there.

To mix C++/WinRT and C++/CX in a C++/WinRT project—including using the `from_cx` and `to_cx` helper functions in the project—you'll need to manually add C++/CX support to the project.

- Create a new C++/WinRT project in Visual Studio using one of the C++/WinRT project templates (see [Visual Studio support for C++/WinRT](#)).
- Turn on project support for C++/CX. In project properties, **C/C++ > General > Consume Windows Runtime Extension > Yes (/ZW)**.

An example C++/WinRT project showing the two helper functions in use

In this section, you can create an example C++/WinRT project that demonstrates how to use `from_cx` and `to_cx`. It also illustrates how you can use namespace aliases for the different islands of code, in order to deal with otherwise potential namespace collisions between the C++/WinRT projection and the C++/CX projection.

- Create a **Visual C++ > Windows Universal > Core App (C++/WinRT)** project.
- In project properties, **C/C++ > General > Consume Windows Runtime Extension > Yes (/ZW)**.
- Add `interop_helpers.h` to the project. For instructions on doing that, see the [from_cx and to_cx functions](#) section above.
- Replace the contents of `App.cpp` with the code listing below.
- Build and run.

`WINRT_ASSERT` is a macro definition, and it expands to `_ASSERTE`.

C++/WinRT

```
// App.cpp
#include "pch.h"
```

```

#include <sstream>

namespace cx
{
    using namespace Windows::Foundation;
}

namespace winrt
{
    using namespace Windows;
    using namespace Windows::ApplicationModel::Core;
    using namespace Windows::Foundation;
    using namespace Windows::Foundation::Numerics;
    using namespace Windows::UI;
    using namespace Windows::UI::Core;
    using namespace Windows::UI::Composition;
}

struct App : winrt::implements<App, winrt::IFrameworkViewSource,
winrt::IFrameworkView>
{
    winrt::CompositionTarget m_target{ nullptr };
    winrt::VisualCollection m_visuals{ nullptr };
    winrt::Visual m_selected{ nullptr };
    winrt::float2 m_offset{};

    winrt::IFrameworkView CreateView()
    {
        return *this;
    }

    void Initialize(winrt::CoreApplicationView const &)
    {
    }

    void Load(winrt::hstring const&)
    {
    }

    void Uninitialize()
    {
    }

    void Run()
    {
        winrt::CoreWindow window = winrt::CoreWindow::GetForCurrentThread();
        window.Activate();

        winrt::CoreDispatcher dispatcher = window.Dispatcher();

        dispatcher.ProcessEvents(winrt::CoreProcessEventsOption::ProcessUntilQuit);
    }

    void SetWindow(winrt::CoreWindow const & window)
    {
    }
}

```

```

winrt::Compositor compositor;
winrt::ContainerVisual root = compositor.CreateContainerVisual();
m_target = compositor.CreateTargetForCurrentView();
m_target.Root(root);
m_visuals = root.Children();

window.PointerPressed({ this, &App::OnPointerPressed });
window.PointerMoved({ this, &App::OnPointerMoved });

window.PointerReleased([&](auto && ...)
{
    m_selected = nullptr;
});
}

void OnPointerPressed(IInspectable const &, winrt::PointerEventArgs
const & args)
{
    winrt::float2 const point = args.CurrentPoint().Position();

    for (winrt::Visual visual : m_visuals)
    {
        winrt::float3 const offset = visual.Offset();
        winrt::float2 const size = visual.Size();

        if (point.x >= offset.x &&
            point.x < offset.x + size.x &&
            point.y >= offset.y &&
            point.y < offset.y + size.y)
        {
            m_selected = visual;
            m_offset.x = offset.x - point.x;
            m_offset.y = offset.y - point.y;
        }
    }

    if (m_selected)
    {
        m_visuals.Remove(m_selected);
        m_visuals.InsertAtTop(m_selected);
    }
    else
    {
        AddVisual(point);
    }
}

void OnPointerMoved(IInspectable const &, winrt::PointerEventArgs const
& args)
{
    if (m_selected)
    {
        winrt::float2 const point = args.CurrentPoint().Position();

        m_selected.Offset(

```